

The Impact Engine: Orchestrating AI, Code, and Decisions

This guide is a no-nonsense, practical playbook for turning messy, real-world problems into testable, scalable systems. Here is exactly what is inside:

- **Section 1: Foundations:** How to define a clear problem gap, map out a 4-phase workflow, and set strict boundaries for where AI should (and shouldn't) be used.
 - **Section 2: Perception & Leverage:** How to reframe problems and find the highest-impact "leverage points" (like code, data, or distribution) to give your system an unfair advantage.
 - **Section 3: Deconstruct the Problem:** How to interview users, prioritize the biggest pain points using the 80/20 rule, and break complex concepts down into bite-sized engineering tasks.
 - **Section 4: Root Cause & Debugging:** Tools like the "5 Whys" and Fishbone diagrams to find out exactly why a pipeline or system broke so you can fix it permanently.
 - **Section 5: Generating Solutions:** Brainstorming frameworks (like the Six Thinking Hats) and how to safely orchestrate local AI models to help generate and filter new ideas.
 - **Section 6: Decision Making:** How to use decision matrices and expected value math to confidently pick the best solution while managing risk and uncertainty.
 - **Section 7: Implementation & Scaling:** The exact steps to ship your code safely, set up monitoring (metrics and alerts), and build automated feedback loops.
 - **Section 8 & 9: Mastery & Action:** Quizzes to test your reflexes, frameworks for teaching this to others, and a 90-day sprint to put it all into action on a real project.
-

Section 1 — Foundations of Systems & Problem Solving

Straight to the point: this section gives you the mental tools, artifacts, and first-mini systems you must be able to produce on demand. Read, practice, and produce the deliverables below — don't just consume them.

1) Intro to Problem Solving — what matters

Goal: turn messy human problems into testable, iterable systems.

Core ideas:

- Problems are *gaps* between current state and a desired outcome. Always quantify both sides (what is, what should be).
- A good solution is measurable, repeatable, and resilient to small changes in context.
- Systems-thinking beats point-solutions: invest in software/data/processes that compound value (less manual effort over time).

Practical deliverable:

- One-page Problem Statement (template below). Produce it for one real problem you want to solve.

One-page Problem Statement template

- Title
 - Stakeholder(s) (who)
 - Current state (data + metrics)
 - Desired state (targets, timeframe)
 - Constraints (budget, time, regulation)
 - Primary metric of success (single number)
 - Secondary metrics (2–3)
 - Quick risks & mitigations (top 3)
-

2) The 4 Phases of Problem Solving — explicit workflow

Use these every time. Treat them like an engineering lifecycle.

1. Discover (Identify & Empathize)

- Output: validated problem statement + success metric.
- Methods: interviews, shadowing, logs, quick telemetry, sample size that proves the problem exists.

2. Formulate (Breakdown & Hypothesize)

- Output: decomposed sub-problems and acceptance criteria.
- Methods: 5 Ws, Feynman breakdown, dependency map, Pareto prioritization.

3. Deliver (Design & Implement)

- Output: minimally viable system that directly moves the primary metric.
- Methods: design docs, sequence diagrams, incremental releases, human-in-loop for risky parts.

4. Validate & Operate (Measure, Learn, Iterate)

- Output: monitored system, rollbacks, decision: continue/pivot.
- Methods: A/B tests, canary, telemetry, SLOs, incident postmortems.

Checklist to run each phase (mini-SDLC):

- Inputs needed
- Success criteria (numeric)
- Deliverables (artifacts)
- Timebox (how long before decision)
- Owner and reviewers

3) Systems-First Thinking (Advanced)

Mindset: you do not ship features; you design *systems* — people + code + data + incentives that compound value.

Principles:

- **Modularity**: small, replaceable components with clear contracts (APIs, message formats).
- **Observability-first**: design telemetry and logs *before* building core logic.
- **Backwards recovery**: assume failures; build robust defaults and rollback paths.
- **Composability**: prefer building blocks that enable multiple future flows (ETL, feature store, auth).
- **Leverage control**: own the pipes that matter (data schemas, distribution channels, billing flow).

Practical architecture checklist:

- Identify the *control plane* (what you must own to change behavior quickly).
- Identify *data contracts* (schema, validation, versioning).
- Define *human-in-loop* boundaries (who can override, how).
- Instrumentation plan (events, metrics, traces) with SLIs and SLOs.
- Security & privacy plan (access control, data retention, encryption).

Deliverable to practice:

- Draw a 1-page system diagram for a simple idea (e.g., a question-answer generator for students): components, data flow, human touchpoints, metrics at each boundary.

4) AI as a Component — clear boundaries and responsibilities

Treat AI like any other subsystem: probabilistic, expensive to operate, and brittle under distribution shift.

Decision checklist — use AI only if:

- The task is ambiguous, language-based, or requires pattern generalization that rules fail to capture.
- You have tolerances for probabilistic outputs (or a human can verify quickly).
- There is signal (data) to train/monitor and a path to improve performance over time.

When NOT to use AI:

- Deterministic, auditable tasks requiring exact correctness (billing, legal eligibility).
- High-stakes safety-critical decisions without human oversight or regulatory approval.

Design patterns for AI-as-component:

- **Predict-then-action:** model predicts; deterministic logic decides action with rules and thresholds.
- **Human-in-loop:** model suggests; human confirms for high-risk outputs.
- **Ensemble & sanity checks:** multiple models + heuristics + exact checks to catch hallucinations.
- **Staged rollout:** test on internal users → beta → production; measure drift and rollback thresholds.

Operational controls to build:

- **Calibration & confidence:** expose model confidences and tune action thresholds.
- **Guardrails:** input sanitization, output filters, and “I don’t know” responses.
- **Audit trail:** store inputs, outputs, model version, prompt template, and decision made.
- **Monitoring:** distribution shift detection, performance by cohort, false-positive/negative tracking.
- **Explainability:** simple, reproducible rationales for outputs when needed.

Ethics & compliance quick list:

- Consent, minimal data collection, de-identification, capability to delete user data.
- Bias assessment and mitigation plans for major cohorts.
- Escalation path for harm reports and clear human accountability.

5) Concrete exercises (do these, one-day sprints)

Sprint A — Problem statement & metric (2–4 hours)

- Pick a problem you care about.
- Produce the one-page Problem Statement and primary metric.

- Run 3 quick user interviews or inspect 100 lines of logs to validate.

Sprint B — 4-phase plan (half day)

- For the same problem, write a 4-phase plan with owners, deliverables, and timeboxes.
- Produce the “go/no-go” criteria at the end of each phase.

Sprint C — System sketch + AI boundary (1 day)

- Draw the system diagram (components + data flows).
- Mark exactly where an AI would sit, what inputs it consumes, what deterministic checks surround it, and when humans must be involved.

Sprint D — Observability & failure plan (half day)

- Define 5 metrics (primary + 4 SLIs).
- Define alert thresholds and an incident playbook (who does what on alert).

6) Measurable outcomes — what mastery looks like

You can claim mastery when you can, repeatedly and in under 72 hours:

- Produce a validated problem statement with measurable primary metric.
- Design a one-page system diagram with AI boundary and monitoring plan.
- Deliver a minimal prototype that moves the metric (even via manual “wizard of oz”).
- Run an A/B or canary and interpret metrics to continue/pivot.

7) Common failure modes and how to avoid them

- Building for hypothetical users — avoid by interviewing and measuring real usage.
- Treating AI as magic — avoid by explicitly defining acceptance criteria and test datasets.
- No observability — avoid by instrumenting first and shipping later.
- Tight coupling — avoid by defining clear APIs and contracts.
- No rollback plan — avoid by making small releases and keeping feature flags.

8) Artifacts and templates you must produce for Section 1

1. One-page Problem Statement (completed for at least one live problem).
2. 4-Phase Plan (document with timeboxes and success criteria).
3. System Diagram (single page).
4. Monitoring matrix (primary metric, 3 SLIs, alert thresholds).
5. Incident playbook (one page).

Deliver each as separate files you can show or iterate on.

9) Suggested readings & study path (order + why)

1. *Thinking in Systems* — systems mental models, leverage points.
2. *Designing Data-Intensive Applications* — data architecture, reliability, and scaling patterns.
3. *The Lean Startup* — build-measure-learn loop and rapid validation.
4. Short papers/articles on human-in-loop and calibration for ML systems (scan when you reach AI component).

(These are stable, classical resources — read summaries first, then deep dive on the ones that change how you design systems.)

10) Time allocation & checkpoint

Suggested minimum: 2 weeks focused practice

- Days 1–3: Problem statements + interviews + Sprint A
- Days 4–7: 4-phase planning + Sprint B
- Week 2: System diagram + AI boundary + Sprints C & D + produce artifacts

Checkpoint deliverable for me (when you're ready): upload your one-page Problem Statement, system diagram (image or text), and your monitoring matrix. I'll critique and convert them into a prioritized implementation plan.

Section 2 — Perception, Mindset, and Leverage

This section trains how you see problems and where you press to win. If Section 1 taught you the mechanics, Section 2 trains the operator: your perception, decision habits, and where you place durable leverage so systems you build compound value.

1) Big-picture summary — what matters and why

- Mindset changes *how* you frame problems, choose metrics, and tolerate uncertainty.
 - Good perception finds the real gap (not the symptom). Good leverage chooses a small number of high-impact knobs (code, data, distribution, incentives) to control.
 - The intersection of perception + leverage = ability to design systems that are robust, adaptable, and owned.
-

2) Core concepts (short definitions)

- **Problem = gap.** Precisely: $\text{Problem} = f(\text{current_state}, \text{desired_state}, \text{constraints}, \text{time})$. Always quantify both states.
 - **Reframing.** Change the formulation of the problem to reveal different solutions. (E.g., “increase retention” → “reduce first-week friction”.)
 - **First principles.** Strip assumptions and reason from fundamentals rather than analogies.
 - **Inversion.** Ask “what would make this fail?” to surface hidden risks.
 - **Leverage.** Any small change that produces outsized impact. Types: software, data, distribution, ownership, pricing, incentives.
 - **Optionality.** Design paths so you can change course cheaply when reality differs from assumptions.
-

3) Cognitive biases & traps to watch

- **Solution bias:** jumping to a familiar tech before validating the problem.
- **Confirmation bias:** seeking data that confirms your plan.
- **Sunk-cost fallacy:** continuing a failing approach because you invested time.
- **Status-quo bias:** preferring existing processes over objectively better changes.
- **Overfitting to early users:** optimizing for vocal early users who are not representative.

Mitigation: short timeboxes, pre-defined go/no-go criteria, blinded metrics where possible.

4) Adopt the right mindset — practical, repeatable habits

- **Start with a measurable gap.** If you can’t write the current and desired states in numbers, you’re not ready to build.
- **Bias to quick, minimum experiments.** Prototype with humans-in-the-loop (Wizard-of-Oz) before building automation.
- **Make decisions probabilistically.** Think in expected value; always note uncertainty and alternative actions.
- **Make default action “test and measure”, not “build”.** Prefer experiments that falsify hypotheses.
- **Ship observability first.** You cannot improve what you can’t measure.
- **Institutionalize reviews.** Weekly 15–30 minute data reviews with defined owners and metrics.

Daily micro-routine (10–20 min): 1) read a single metric vs target; 2) write one assumption you want to test today; 3) one micro-action to reduce uncertainty.

5) Exercises — move this from theory to reflex

A — 5-minute reframe (do daily for a week)

1. Choose one problem you're stuck on.
2. Write current state (1 sentence + one number) and desired state (1 sentence + one number).
3. Reframe it three different ways (e.g., customer, product, business, operations perspective).
4. Pick the reframing that exposes the smallest experiment you can run.

Timebox: 5 minutes. Deliverable: 1 short note.

B — Problem gap write-up (1 hour)

- Title, stakeholder, current_state (quantified), desired_state (quantified), one primary metric, 3 assumptions.
- For each assumption, list a 48–72 hour experiment to test it.

Deliverable: one-page gap write-up.

C — Pre-mortem + inversion (45 minutes)

- Imagine the project failed 6 months from now. List 10 reasons. For the top 3, write a mitigation/test you can run in 2 weeks.

D — Leverage mapping (90 minutes)

- For your project, list potential leverage points (code, data, distribution, incentives, cost structure). For each: control_level (own/partner/external), switching_cost (0–10), potential_impact (0–10). Prioritize by **impact** / **switching_cost**.

Deliverable: 1-page leverage map & prioritized 2–3 action items.

6) The Architect's Leverage — concrete checklist & template

Leverage types

1. **Code / Platform** — your ability to change product behavior quickly (APIs, modular code, feature flags).
2. **Data** — unique, hard-to-reproduce data sources or labeling pipelines.
3. **Distribution** — channels that let you cost-effectively reach users (email lists, school partnerships, app stores).
4. **Incentives & Processes** — what makes people (users, partners, teammates) behave the way you need.

5. **Ownership & Legal** — IP, contracts, compliance, billing flows that lock in advantages.

Leverage mapping template (spreadsheet columns)

- **leverage_area** (code/data/dist/distribution/incentive/legal)
- **specific_asset** (e.g., question bank, email list)
- **control_level** (own/partner/external)
- **switching_cost** (0–10)
- **potential_impact** (0–10)
- **time_to_build** (days)
- **priority_score** = $\text{potential_impact} / (1 + \text{switching_cost})$
- **action** (next step, owner, due date)

Use this to pick the top 2 lever knobs to move early.

Rules of thumb

- Prioritize assets you can **own** or cheaply acquire exclusive access to (data, billing flow).
- Aim to create at least one *hard-to-copy* asset by month 3 (unique dataset, distribution channel, or workflow integration).
- Avoid betting everything on a single external platform unless you control migration paths.

7) Scoring & decision heuristics for leverage

- If **priority_score** > 5 → high priority.
 - Required guardrail: before investing > 2 weeks, run a 3–point validation: feasibility test, demand test, and cost estimate.
 - If a leverage requires partnership (control_level = partner), build fallback options and a migration plan before committing.
-

8) Example (applied to your Medical-AI roadmap)

NEETPrepGPT

- Leverage candidates: proprietary MCQ dataset (data), exam-scraping automation (code), teacher/referral network (distribution), subscription billing (ownership).
- Quick decision: prioritize acquiring/curating a high-quality MCQ dataset (impact high, switching_cost high later). Run a 7-day data acquisition sprint (target: 5k validated questions).

Symptom2Specialist

- Leverage candidates: access to clinician referral lists (distribution), quality labeled symptom-to-specialist pairs (data), FHIR integration (code/legal).
- Quick decision: focus on a diagnostic routing policy that's human-in-loop (low-cost MVP) while you pursue FHIR partnerships.

9) Deliverables you should produce for Section 2

- Daily journal entries for the 5-minute reframe (7 days).
- One gap write-up (problem + 3 assumptions + experiments).
- One pre-mortem document (10 failure modes + mitigations).
- Leverage map spreadsheet with priority scores and the top 2 actions scheduled.

10) Mastery criteria (how you know you've internalized this)

You can claim mastery when, for a new problem, you can in under 72 hours:

1. Produce a quantified gap write-up with measurable primary metric.
2. Run and report results from at least two experiments that materially reduce the top 3 assumptions.
3. Produce a leverage map that points to 1–2 owned assets and an actionable 2-week plan to build them.
4. Make a go/no-go decision based on data, not belief.

11) Common failure modes & fixes

- **Too many leverage attempts.** Fix: cap to top 2 for the first month.
- **No experiments—only plans.** Fix: require an experiment before any >2-week build.
- **Leverage chosen for prestige, not impact.** Fix: use priority_score formula, not intuition.
- **Mindset = perfectionism.** Fix: enforce timeboxed experiments and accept noisy but directional data.

12) Immediate action (what to do next — do this now)

1. Pick one active problem you care about (student retention, MCQ extraction accuracy, clinician routing accuracy).
2. In 60–90 minutes produce: the gap write-up + leverage map (top 6 rows).
3. Share those two artifacts; I'll convert them into a 2-week experiment plan with owners and metrics.

Section 3 — Identify, Formulate, and Deconstruct

This section trains the highest-leverage skill: turning messy human signals into crisp, testable problem definitions and then breaking those down into parts you can solve reliably. You'll practice active listening, practical framing (5W1H), Pareto prioritization, and the Feynman-style deconstruction that turns domain complexity into engineering workstreams.

1) Goal — what you must be able to do

- Extract the *true* problem from users/observations (not the symptom).
 - Write a compact, unambiguous problem formulation that includes stakeholders, constraints, and the primary success metric.
 - Prioritize the smallest set of causes that drive most of the pain (80/20).
 - Decompose the problem into independent, testable sub-tasks you can assign, measure, and iterate.
-

2) Develop Active Listening — practical playbook

Why it matters: interviews are where assumptions die or get validated. Most founders/engineers hear what they expect to hear.

Core habits (practice until reflex):

1. **Prepare** — 3 questions you must answer from this conversation.
2. **Ask open questions** — start with "Tell me about..." / "Walk me through..." Avoid yes/no.
3. **Pause** — after they speak, count to 3 before responding. People reveal more in the pause.
4. **Paraphrase & confirm** — "So what I heard: X. Is that right?" Use this every 2–3 answers.
5. **Quantify** — push for numbers or concrete examples: "How many times per week?", "Show me the last 3 examples."
6. **Avoid solution language** — don't lead with "Would you like an app that...?" — ask about current behavior first.
7. **Record + timestamp** — take a short transcript/notes and mark quotes you'll use.
8. **End with next step** — ask permission to follow up, and what you'd show them to confirm you understood.

30-second checklist before every interview:

- Goal? (1 line)
- Top 3 questions?
- One hypothesis I want to falsify.

Sample starter questions

- “What does a typical day look like for you when you deal with X?”
- “What is the last time this caused a problem? Tell me what happened step by step.”
- “What workaround do you currently use?”
- “How do you know when it’s solved?” (forces metrics)

Deliverable: 6–8 interview transcripts + a 1-page synthesis (themes, supporting quotes, numbers).

3) The 5 Ws and 1 H — exact template & method

Use 5W1H to reduce ambiguity and surface missing data.

Template (fill for every problem)

- **Who** (stakeholders & actors): primary user, affected parties, decision-makers.
- **What** (the issue): observable behavior, symptom, or event. Use numbers.
- **Where** (context/location): platform, workflow, geographical, device.
- **When** (timing/frequency): when it occurs, seasonality, timelines.
- **Why** (root cause hypothesis): why it happens — list hypotheses ranked by confidence.
- **How** (current workaround / flow): step-by-step current process, tools used, failure points.

Practical rule: if any box is blank or vague, you *do not* build—run targeted interviews or measure logs first.

Example (realistic) — problem: low completion of practice sets in NEETPrepGPT

- Who: 11th–12th NEET aspirants using our app; teachers recommending to small groups.
- What: only 18% complete a 50-question set within 24 hours; average time on set = 12 minutes.
- Where: mobile app; offline mode often used during commute.
- When: drop-off spikes on Sunday evenings and during exam weeks.
- Why: hypothesis 1: questions too long/difficult; hypothesis 2: UI makes navigation slow; hypothesis 3: users lack motivation/feedback loop.

- How: flow = open app → select chapter → start set → face long explanation after each question → abandon.

From this you can design experiments: shorten explanations, add progress feedback, or batch questions.

Deliverable: one filled 5W1H for your current top problem.

4) Pareto (80/20) — how to prioritize the small set of causes

Goal: identify the 20% of issues producing 80% of the pain, then attack them first.

Steps (data-driven):

1. **List candidate causes** (use your interviews + logs).
2. **For each cause, estimate two numbers:** (A) frequency (how often it appears) and (B) impact per occurrence (how costly/important). Use 0–10 scale if numeric data absent.
3. **Compute score = frequency × impact.**
4. **Sort descending** and pick the top ~20% (or top 3) as your focus.

Practical spreadsheet columns:

- `cause` | `evidence` | `frequency_score` (0–10) | `impact_score` (0–10) | `priority_score` | `next_experiment`

Example (for MCQ pipeline):

- cause: low-quality distractors | evidence: 15% flagged by teachers | freq 7 | impact 8 | score 56 → top priority

Quick validation: for top cause, run a 7–14 day targeted experiment and track the primary metric.

Deliverable: Pareto table + one targeted experiment plan for top cause.

5) Domain-Specific Formulation — healthcare & education notes

When working in healthcare/education, the problem framing must explicitly include safety, incentives, and ground truth.

Key differences:

- **Ground truth is expensive** — labels come from clinicians/teachers; plan for labeling cost/time.
- **Human incentives matter** — teachers and clinicians have workflows and liability concerns; your solution must reduce their work or demonstrate clear ROI.
- **Regulatory & privacy constraints** — define data minimization, consent, and retention up front.
- **Evaluation is cohort-based** — performance can vary across age, socio-economic groups, or device types; stratify metrics.
- **Adoption is trust-driven** — early adopters are gatekeepers (school admin, department head, clinician), not only end-users.

Practical checklist when you frame a healthcare/education problem:

- Stakeholder map (who approves, who uses, who pays).
- Label plan & cost (how many expert labels needed, how to bootstrap).
- Safety thresholds (what false-positive/false-negative rates are acceptable and who signs off).
- Operational constraints (offline access, low-bandwidth, language variations).
- Pilot plan (small cohort, human oversight, clear rollback).

Example: Symptom2Specialist

- Who: patients, frontline clinicians, referral coordinators.
- What: incorrect specialist routing causing wasted appointments.
- Ground truth: clinician-labeled routing for 5k cases for initial model.
- Safety: human-in-loop for ambiguous cases; threshold to escalate to clinician review when confidence < 0.7.

Deliverable: domain-safe problem formulation + pilot protocol.

6) Deconstruction — Feynman Technique adapted for systems

Purpose: convert domain complexity into bite-sized, testable engineering tasks.

Step-by-step:

1. **Explain simply** — write one-paragraph explanation of the problem as if for a non-expert (5 sentences max).
2. **List required knowledge** — items you must understand to implement a solution (e.g., exam format, data schema, clinical referral taxonomy).
3. **Break into sub-problems** — each should be independently testable and deliverable in ≤ 2 weeks. Example sub-problems: data ingestion, question-normalization, difficulty-calibration, UI microflow for practice sessions.

4. **For each sub-problem write:** purpose, input(s), output(s), acceptance criteria (numeric), owner.
5. **Identify dependencies** — which sub-problems block others; mark safe reorderings.
6. **Create a minimal “human-in-loop” version** — a Wizard-of-Oz that simulates the final automation so you can validate before building.

Template (one sub-problem per row):

- `name | purpose | inputs | outputs | acceptance_criteria | timebox_days | owner | dependencies`

Example sub-problem: “Question Normalizer”

- purpose: convert scraped questions to canonical format
- inputs: raw HTML question pages
- outputs: JSON `{question, options, answer, difficulty}`
- acceptance: 95% field extraction accuracy on 200-sample test
- timebox: 7 days
- dependencies: none

Deliverable: Deconstruction doc with 6–12 sub-problems, acceptance criteria for each.

7) Exercises — concrete, timeboxed practice

Do these in sequence for one live problem (use NEETPrep or Symptom2Specialist).

A — **Active Listening Sprint** (4 interviews, 2–4 hours)

- Goal: answer 3 prepped questions + extract 3 quotes and 2 numbers per interview.
- Deliverable: 1-page synthesis.

B — **5W1H Fill** (30–60 minutes)

- Fill the template fully. If any cell is empty, run one more interview/log query.

C — **Pareto Prioritization** (1–2 hours)

- List 8–15 causes from interviews/logs and compute scores. Pick top 2.

D — **Feynman Deconstruction** (3–6 hours)

- Produce a sub-problem table with acceptance criteria and a Wizard-of-Oz plan for the highest-priority sub-problem.

E — **Mini-pilot** (7–14 days)

- Run the human-in-loop test that simulates automation and measure the primary metric change.
-

8) Metrics & success criteria (what to measure)

For Section 3 mastery, measure:

- Time-to-validated-problem (target: ≤ 72 hours from selection to filled 5W1H).
 - Interview yield: average number of concrete, measurable insights per interview (target ≥ 2).
 - Pareto hit: % of measured pain explained by top 2 causes (target $\geq 60\%$).
 - Sub-problem readiness: number of sub-problems with numeric acceptance criteria (target ≥ 6).
 - Pilot delta: percent change in primary metric during Wizard-of-Oz pilot (target depends on problem; aim for directional $\geq 10\text{--}20\%$).
-

9) Common failure modes & fixes

- **Shallow interviews** — fix: enforce open questions, paraphrase confirmations, request examples.
 - **Vague metrics** — fix: force numbers into current/desired state before moving on.
 - **Over-decomposition** (too many tiny tasks) — fix: group into deliverables that move the main metric.
 - **Ignoring incentives** — fix: map stakeholder incentives early and reframe solutions to align them.
 - **Building before validating** — fix: require a Wizard-of-Oz pilot for every major automation.
-

10) Readings & tools (fast track)

- Short: *How to Talk to Users* (articles on user interviews).
 - Pareto primer: short explainer + spreadsheet template (search “Pareto analysis template”).
 - Feynman technique: quick guide + example deconstruction.
 - Tools: Otter/Rev for transcripts, a simple Notion/Google Sheet for 5W1H & Pareto, Loom for recording demos of user flows.
-

11) Deliverables you must produce for Section 3

1. 6–8 interview transcripts + 1-page synthesis.
2. Filled 5W1H for your target problem.

3. Pareto table with scores and top 2 prioritized causes.
4. Deconstruction spreadsheet (6–12 sub-problems with acceptance criteria).
5. Wizard-of-Oz pilot plan and 7–14 day pilot results.

Share these artifacts and I will convert them into a prioritized sprint backlog with exact experiments, data checks, and monitoring hooks.

Section 4 — Root Cause Analysis & Debugging

Short version: stop treating incidents as surprises. Build a disciplined, repeatable process to find the *real* root cause quickly, fix it safely, and prevent it from returning. Below is a compact, practical playbook you can apply to product bugs, infra failures, data issues, scrapers, or ML pipelines.

1) Core principles (always follow)

- **Measure first, act second.** Gather the minimal evidence before making irreversible changes.
 - **Isolate to narrow blast radius.** Reproduce in staging or shadow mode where possible.
 - **Prefer rollback over risky patches.** Restore service quickly, then debug.
 - **Instrument before optimizing.** If you can't observe it you can't fix it reliably.
 - **Root cause = deepest systemic reason.** Not the immediate symptom (e.g., "worker crashed" vs "new library changed behavior and broke retry logic").
-

2) The 5 WHYs — exact method + template

Method: start with the symptom; ask "Why?" five times (or until you hit an organizational/process-level cause). Stop when you reach a cause you can act on.

Template (fill these during the RCA)

1. Symptom (what you saw, when, scope)
2. Why #1 → Answer
3. Why #2 → Answer
4. Why #3 → Answer
5. Why #4 → Answer
6. Why #5 → Answer (root cause candidate)
7. Short-term fix (how you immediately fixed or mitigated)

8. Long-term corrective action (prevent recurrence)
9. Owner + due date

Example — NEETPrepGPT scheduled MCQ ingestion failure:

- Symptom: ingestion job failed at 02:15; 100% of questions missing for today.
- Why 1: Job exited with JSON parse error.
- Why 2: Input file contains HTML `<pre>` wrapper, not JSON array.
- Why 3: Scraper started returning HTML because website responded with anti-bot page.
- Why 4: Scraper used a header that revealed bot fingerprint; site served anti-bot page.
- Why 5 (root): No rotating proxy / no captcha handling and code assumed stable HTML — gap in scraper resilience.
- Fix: roll back to previous day's dataset, retry job with cached files.
- Preventive: add proxy rotation, add sanity check for HTML vs JSON, add test harness. Owner: dev A, due in 3 days.

3) Fishbone (Ishikawa) — structured visualization

Use fishbone to list causes across categories and then score/prioritize them.

Typical categories for software/data systems:

- People (ownership, access, training)
- Process (release, testing, runbooks)
- Equipment (servers, network, proxies)
- Data (schema drift, corrupt data, missing fields)
- Code (regressions, dependency changes)
- External (third-party APIs, rate limits, legal)

How to use:

1. Draw the fishbone with the symptom on the right.
2. Populate causes under categories from interviews/logs.
3. For each cause add evidence and an estimated probability (0–1).
4. Convert top candidates into targeted experiments or 5 Whys threads.

Exercise: run a 30–60 minute fishbone session with the engineering + ops owner, product person, and one domain expert.

4) Suggested solution pattern (incident lifecycle)

1. **Triage (0–15 min)**

- Severity (S1/S2...), blast radius (users affected), business impact.
- Who owns incident channel and communication.

2. Mitigate (15–60 min)

- Quick mitigation: toggle feature flag, scale replicas, redirect traffic, restore backup.
- Record actions (time + actor).

3. Gather evidence (parallel)

- Logs, trace spans, recent deploy IDs, config diffs, metrics, database counts.
- Take snapshots: `SELECT count(*)` checks, head of queues, recent error traces.

4. Diagnose (1–4 hours)

- Use 5 Whys and fishbone; reproduce in staging if possible.

5. Fix (4–24 hours)

- Prefer safe fixes: config change or rollback. If patching, write tests first and canary the change.

6. Postmortem (within 48 hours)

- Publish timeline, root cause, permanent fix, and owner for prevention.

7. Close & follow-up

- Implement long-term changes, update runbooks, write tests, and add monitoring.

5) Pipeline diagnostics — async workflows, scraping, ingestion

Below are diagnostics patterns and checks you should run. Use them as a checklist.

A — General observability for pipelines

- Unique **trace ID** per request/message preserved across services.
- Structured logs: `{timestamp, service, trace_id, severity, event, payload_hash}`.
- Metrics: throughput (items/s), success_rate, error_rate, latency P50/P95/P99, queue_length, consumer_lag.
- Data freshness metric: `max(now - ingestion_time)` and per-source freshness.

B — Async workflows (message queues, workers)

Symptoms: consumer lag, retries, poison messages, duplicate processing.

Checks:

- Consumer lag (`kafka-consumer-groups` or cloud console).
- Dead-letter queue (DLQ) counts.
- Retry policy: exponential backoff configured? max attempts.
- Idempotency keys present? (message_id dedupe).
- Checkpoint/commit offsets (are consumers committing?).
- Backpressure signals: queue length vs processing capacity.

Quick SQL-like checks (example):

- `SELECT count(*) FROM events WHERE ingestion_time > now() - interval '1 hour';`
- Check duplicates: `SELECT message_id, count(*) FROM events GROUP BY 1 HAVING count(*) > 1;`

Fixes:

- Move poison messages to DLQ manually, inspect payload.
- Reduce concurrency until a hotfix deployed.
- Add idempotency or stronger validation to reject bad messages early.

C — Web scraping scripts

Symptoms: fewer items scraped, parse exceptions, sudden content change.

Checks:

- HTTP status codes distribution (200, 403, 429, 500).
- Response body length / content-type changes.
- CAPTCHA/anti-bot HTML signatures.
- Recent deploy or header changes in scraper.
- Proxy IP blacklist hits.

Diagnostic commands/steps:

- Re-run failing URL in headful browser to compare.
- Compare CSS/XPath selectors on sample pages.
- Hash response body to detect changed templates: `sha256sum response.html`.
- Check rate-limit headers (`Retry-After`).

Fixes:

- Add selector-robustness: fallbacks, heuristics, ML extraction.
- Implement proxy rotation / throttle / exponential backoff.
- Alert on >X% unexpected HTML responses.

D — Data ingestion & schema drift

Symptoms: nulls, malformed JSON, downstream model failures, feature mismatches.

Checks:

- Schema versioning header/field present? `schema_version` in payload.
- Field-level null/NaN rates vs baseline.
- Distribution shifts (value histograms) vs reference.
- Ingested row counts by source/time.

Sample checks:

- `SELECT field, count(*) FILTER (WHERE field IS NULL) FROM table WHERE ts > ... GROUP BY field;`
- Compute checksums of input batches and compare with previous runs.

Fixes:

- Add schema validation (Avro/Protobuf/JSON Schema) on ingestion.
- Reject/alert on unexpected schema changes.
- Add backward-compatible transformations and migration scripts.

6) Observability & alerts — what to instrument now

Minimum SLI set for pipelines:

- **Success rate** (items processed / items received) — alert if < 99% over 5m.
- **Throughput** — alert on sudden drop >50% vs baseline.
- **Queue length / consumer lag** — alert if > X or increasing trend for 15m.
- **Data freshness** — alert if max age > threshold (e.g., 60 minutes).
- **Error rate** (exceptions per 1k items) — alert on spike.
- **Schema validation failures** — alert on any non-zero.

Example alert thresholds (start conservative, iterate):

- Success rate < 99% for 5m → page on-call.
- Queue lag > 5k messages for 15m → warn, >10k → page.
- Data freshness > 2× SLA → page.

7) Postmortem template (publishable)

- Title + severity
- Dates/times (detection → mitigation → resolution) with UTC timestamps
- Summary for execs (2–3 lines)
- Impact (users affected, downtime, financial/clinical impact)
- Timeline (ordered events with times)
- Root cause (5 Whys conclusion)
- Contributing factors (from fishbone categories)
- Immediate remediation (what you did to restore)
- Long-term fixes (owners + deadlines)
- Action items & verification plan (how will you confirm it's fixed)
- Lessons learned

Make a habit: every postmortem finishes with at least one automation or test that makes the same human action unnecessary.

8) Exercises (practice sprints)

A — Fishbone workshop (60–90 min)

- Pick a recent incident. Run fishbone with 3 people, produce top 3 hypotheses and owners.

B — 5 Whys drill (30–45 min)

- For each hypothesis, run 5 Whys and produce an action (fix or experiment).

C — Pipeline hunt (2–4 hours)

- Produce an “ingestion health dashboard” with the minimum SLIs above and run a red-team test: simulate schema drift and confirm alerts.

D — Scraper resilience test (1 day)

- Randomly modify a sample page template and verify your scraper detects and alerts.

E — Postmortem write + automation sprint (2 days)

- Write a postmortem for a simulated incident and implement one automation (schema validation rule or DLQ alert).
-

9) Common failure modes & how to avoid them

- **No observability** → instrument before deploying.
- **Blame game** → focus on systems and process fixes, not people.

- **No ownership** → every pipeline has a named owner with SLAs.
 - **One-off manual fixes** → convert them to automated tests or scripts within 7 days.
 - **Silent drift** → schedule daily or hourly freshness checks for critical flows.
-

10) Deliverables & mastery criteria

You're competent when you can, in ≤ 24 hours for a critical failure:

1. Run triage and restore service (rollback or mitigation).
2. Produce a 5 Whys + fishbone summary with a confident root cause.
3. Publish a postmortem with at least two permanent fixes and assign owners.
4. Add instrumentation or automated checks that would have detected the issue earlier.

Deliverables you should keep in the repo:

- Incident runbook (playbook).
 - Postmortem template and archived postmortems.
 - Health dashboard with SLIs.
 - Test harness for scrapers and ingestion with schema checks.
-

11) Immediate actions you can do now (30–120 min)

1. Create a simple "health check" script that verifies: last ingestion timestamp, count in last hour, number of schema failures. Hook it to alerting.
 2. Add trace IDs to the top 3 critical flows and confirm they propagate to logs and metrics.
 3. Draft the incident runbook for your highest-risk pipeline (who to ping, where to rollback, how to restore).
 4. Run a fishbone + 5 Whys on the largest recurring error in your logs and assign corrective actions.
-

Section 5 — Solution Generation & Orchestration

This is where your ideas become options you can actually test and ship. The goal: produce many *high-quality* candidate solutions, evaluate them quickly, and orchestrate human + model workflows so the best options get implemented and improved over time.

1) Short summary — what matters

- Quantity of ideas matters only until quality filters are in place. Generate widely, then rapidly converge.
- Use structured, repeatable facilitation methods (Six Thinking Hats, SCAMPER, analogies) so ideation isn't random.
- Visualize relationships with mindmaps and decision artifacts so ideas are composable into systems.
- Use local/offline models as scalable ideation assistants — but orchestrate them with deterministic filters, ensembles, and humans-in-the-loop to avoid hallucinations and repetition.

2) Techniques for lateral thinking (practical list)

Use these intentionally in separate steps of a session.

1. **SCAMPER** — Substitute, Combine, Adapt, Modify, Put to another use, Eliminate, Reverse. Apply to your current solution to generate variants.
2. **Forced analogies** — Pick an unrelated domain (restaurant, airline, chess). Ask: "How would a restaurant solve this?" Translate insight back.
3. **Constraint flip** — Reduce a key resource (time, memory, money) by 10x and force solutions that work under that constraint.
4. **Reverse assumptions** — List top 5 assumptions and invert each: "What if opposite were true?"
5. **Random stimulus** — Use an unrelated word/image to seed novel metaphors; force a 2-minute mapping to the problem.
6. **Morphological matrix** — List dimensions and options, then combine columns to create designs.
7. **Analogical transfer** — Map a proven process from a different industry and adapt the mechanics (e.g., subscription -> education cohorts).

Use 2–3 techniques per brainstorming block (20–40 minutes).

3) Six Thinking Hats — exact facilitation script

Purpose: make the group switch perspectives systematically so no dimension is ignored.

Hats & what they do:

- **White (facts)** — What do we know? Data, constraints, missing info. (5–7 min)
- **Red (emotion)** — Gut reactions and intuitions. No justification required. (3–5 min)
- **Black (critical)** — Risks, why it may fail. (5–7 min)
- **Yellow (optimistic)** — Benefits and why it could work. (5–7 min)
- **Green (creative)** — New ideas, lateral moves. (10–15 min)

- **Blue (process)** — Meta: summarize, pick next steps, prioritize. (5–10 min)

Facilitation script (60–90 min session):

1. Blue: state goal + metric (2 min).
2. White: quick data readout (5 min).
3. Red: round-robin gut reactions (3–5 min).
4. Black: list top risks (5 min).
5. Yellow: list opportunities (5 min).
6. Green: 2 rounds of rapid idea generation using SCAMPER + forced analogies (15 min).
7. Blue: cluster ideas, pick top 6 for validation (5–10 min).
8. Voting: weighted vote (each person has 10 points to allocate) (5 min).
9. Assign experiments for top 2 ideas (10 min).

Deliverable from session: ranked idea list + 2 experimental hypotheses with owners and 7–14 day experiments.

4) Mindmap — how to build one that actually helps

Mindmaps = the canonical way to turn brainstorm blobs into architecture.

Steps:

1. Center node = problem statement (1 line + primary metric).
2. First ring = solution families (UI, data, incentives, partnerships, ML, operations).
3. Second ring = components for each family (e.g., under ML: model, dataset, CI, latency guardrails).
4. For each component add: inputs, outputs, acceptance criteria, owner, timebox.
5. Use color to indicate feasibility (green = can do in <2 weeks, amber = 2–8 weeks, red = >8 weeks or requires partnership).
6. End with three immediate actions: experiment A, experiment B, research spike.

Tools: Miro/Excalidraw/Obsidian graph — but a simple whiteboard or paper is fine for one-person runs.

Mindmap exercise (60–90 min):

- Produce a one-page mindmap for a chosen idea, then translate the top left branch into a 7-day sprint backlog.

5) Orchestrating local models for ideation (advanced, practical)

Why local models?

- Confidentiality, cost control, offline work, repeatability. They're great for churn-heavy ideation and synthesis.

A robust orchestration pattern (seed → variants → aggregator → filters → human):

1. **Seed context** — canonical brief: problem statement (1 line), constraints, metric, 5 facts, top 3 failures to avoid. Keep ≤ 200 tokens for each run.
2. **Variant generation stage** — run multiple model instances or prompts to produce N variants (N = 5–20). Use temperature diversity and prompt templates that force structure (title, one-sentence idea, 3 bullet pros, 3 bullet cons, risk level).
3. **Automated filters** — immediate filters: duplicates (semantic similarity), toxic content, impossible constraints, or known banned patterns.
4. **Scoring/ensemble** — compute simple heuristics per variant (impact_estimate, implementation_complexity, novelty_score). For novelty use embedding cosine distance vs existing idea bank.
5. **Human triage** — present top K (3–6) to humans with the mindmap branch and a suggested next experiment.
6. **Human-in-loop synth** — humans refine prompts or seed additional rounds.
7. **Logging & audit** — save model version, prompts, outputs, embeddings, and human decisions (audit trail).

Simple orchestration pseudocode (conceptual)

```
seed = build_seed(problem, constraints, metrics)
variants = []
for temp in [0.2, 0.6, 1.0]:
    variants += run_model(seed, temperature=temp, n=5)
variants = dedupe(variants, threshold=0.85)
scored = score_variants(variants) # heuristics + embeddings
top = select_top(scored, k=6)
present_to_humans(top)
```

Prompt patterns (high-quality)

- “You are an experienced product designer for low-bandwidth mobile apps. Given: [seed]. Produce: (1) name, (2) one-line description, (3) three concrete user flows, (4) three measurable acceptance criteria (numbers).”
- Ask models to explain **assumptions** and **required resources** explicitly.

Safety & quality rules

- Never let model outputs directly become production artifacts without human review.
- Always require models to list assumptions and failure modes.
- Track model version and seed prompt; archive outputs for reproducibility.

Practical orchestration tips

- Run multiple lightweight prompts rather than one long prompt — diversity beats length.
- Use embeddings to detect novelty and avoid redundant ideas.
- Maintain an “idea bank” with metadata (source, date, tests run, outcome) so you don’t repeat failed experiments.

6) Exercises & sprints (concrete)

Sprint A — Rapid ideation + mindmap (3 hours)

- Prepare: one-line problem + metric.
- Run a 90-minute Six Hats session (solo or group).
- Create mindmap and pick top 3 idea clusters.
- Output: mindmap PNG + ranked idea list.

Sprint B — Local-model assisted ideation (2–4 hours)

- Build seed doc (200 tokens).
- Run 3 prompt templates across 2 temperature settings, gather 15 variants.
- Dedupe and score; present top 5 in a short report.
- Output: top 5 ideas + prompts used + model metadata.

Sprint C — Experiment design (4 hours)

- For the top idea, create 2-week experiment with owner, metric, acceptance criteria, and data collection plan.
- Run a pilot (wizard-of-oz) if automation not ready.

7) Evaluation & convergence (how to pick)

- Use a **Decision Matrix** with these axes: expected impact (0–10), ease of implementation (0–10), risk (0–10), time to learn (days). Compute $\text{score} = \text{impact} \times (\text{ease} / (1 + \text{risk}))$. Pick highest scores.
 - Use weighted voting in groups (each participant has 10 points).
 - Require that top idea has a clear 7–14 day experiment and a measurable primary metric.
-

8) Deliverables you should produce for Section 5

- Facilitation script for Six Thinking Hats tailored to your team.
 - One mindmap per problem (exportable image).
 - Idea bank records with embeddings + scores.
 - Two model prompts used for ideation + provenance (model version, temps).
 - 7–14 day experiment plan for the top idea.
-

9) Mastery criteria

You're competent when, under 72 hours, you can:

1. Run a structured ideation session and produce a ranked backlog of at least 6 ideas.
 2. Use a local/offline model to produce at least 10 diverse idea variants, filter them to 5 novel candidates, and pick 2 to experiment on.
 3. Translate one chosen idea into a 14-day experiment with clear metric and data collection.
-

10) Common failure modes & fixes

- **Ideas without tests** — require an experiment before any >2-week build.
 - **Groupthink** — force anonymous idea submission and use embeddings to dedupe clone ideas.
 - **Overreliance on models** — always require human validation and explicit assumptions from the model.
 - **Flood of low-quality ideas** — cap variants per prompt, enforce structure in the output, and score automatically.
 - **No provenance** — log model version, seed, and human decision for reproducibility.
-

11) Quick templates (copy-paste)

Seed brief (copyable)

```
Problem: [one line]
Primary metric: [metric + baseline + target]
Constraints: [top 3 constraints]
Top 3 failures to avoid: [list]
Users & stakeholders: [who]
Timebox: [days]
```

Model prompt template (copyable)

You are an experienced designer for [domain]. Given:
[seed brief]

Produce 8 distinct idea variants. Each variant MUST include:

- 1) Title (<=6 words)
- 2) One-line summary
- 3) 3 concrete user flows (bulleted)
- 4) 3 acceptance criteria (numeric when possible)
- 5) 2 main assumptions
- 6) Estimated time to prototype (days)

Return as a JSON array.

Idea scoring quick sheet

- impact (0–10) | ease (0–10) | risk (0–10) | novelty (0–10)
- score = impact * (ease / (1 + risk))

Section 6 — Evaluation, Uncertainty, & Selection

Short version: generate options, score them with transparent math, measure uncertainty, and pick decisions that maximise expected value *and* preserve optionality. Don't pretend you know more than you do — quantify uncertainty, run cheap experiments to reduce it, and design architectures with fallbacks.

1) Core ideas (in one line each)

- **Decision = tradeoff under uncertainty.** Every choice trades upside, cost, and risk.
 - **Make tradeoffs explicit with numbers.** Use impact, effort, risk, and time-to-learn as axes.
 - **Use expected value (EV) + robustness.** Prefer options with good EV *and* limited downside.
 - **Reduce uncertainty cheaply.** Run experiments where the *expected value of information* exceeds the experiment cost.
 - **Keep optionality.** Prefer architectures that let you pivot cheaply if reality differs.
-

2) Practical decision tools (what to use, when)

1. Advantages / Disadvantages (quick filter)

- Fast, qualitative. Use when you need a quick yes/no in <30 minutes.

2. Weighted vote (group prioritization)

- Use when you need group buy-in and a simple ranking. Everyone allocates points; sum them.

3. Decision matrix (numerical scoring)

- Core tool for comparing options across multiple criteria with weights.

4. Decision tree + EV calculation

- Use when outcomes are sequential and probabilistic (choose A now → chance of success → follow-up choice).

5. Robust-decision rules (minimax, maximin, regret)

- Use when downside is catastrophic or probabilities are unreliable.

6. Value of Information (VOI / EVPI)

- Use to decide whether to run an experiment before committing to a build.

3) Decision matrix — exact template & formula

Columns for spreadsheet:

option | criterion1 | ... | criterionN | weights | weighted_score | total_score | rank | notes

Common criteria and how to score (0–10):

- **impact** (expected benefit to primary metric)
- **ease** (implementation simplicity)
- **cost** (negative score or flip sign)
- **time_to_value** (shorter preferred)
- **risk** (inverse: lower risk → higher score)
- **ethical_concern** (higher score = less concern)

Formula (for each option):

1. Normalize each criterion to 0–10.
2. Choose weights w_i (sum to 1).
3. $\text{weighted_score} = \text{sum}_i (w_i * \text{score}_i)$

4. $\text{total_score} = \text{weighted_score}$ — sort descending.

Example: Three criteria with weights: impact 0.5, ease 0.3, risk 0.2. If option A: impact=8, ease=6, risk=7:

- $\text{weighted} = 0.58 + 0.36 + 0.2 \cdot 7$
- $= (0.58) + (0.36) + (0.2 \cdot 7)$
- $= 4.0 + 1.8 + 1.4$
- $= 7.2 \text{ total_score}$

(Notice the digit-by-digit multiplication and addition.)

Use a separate column for **sensitivity**: how much the score changes if a key input is $\pm 30\%$.

4) Weighted vote — quick rules

- Each participant gets 10 points (or 100) to allocate among options however they like.
- Tally points to get a ranked list.
- Use when group dynamics would otherwise let a single voice dominate.
- Follow-up: take top 2–3 options and apply decision-matrix/EV analysis.

5) Decision trees & expected value — practical how-to

Decision tree nodes:

- Decision nodes (square) — you choose an action.
- Chance nodes (circle) — probabilistic outcomes (with probabilities).
- Terminal nodes — payoffs/costs.

EV calculation example (simple, step-by-step):

- Option A: build feature now. Cost = ₹100,000. If succeeds (prob $p=0.4$) payoff = ₹300,000 (net profit = ₹300,000 – ₹100,000 = ₹200,000). If fails (prob $1-p = 0.6$) net = –₹100,000.
- $\text{EV} = p * +₹200,000 + (1-p) * (-₹100,000)$.
 - Compute $p * 200,000 = 0.4 * 200,000 = 80,000$.
 - Compute $(1-p) = 0.6$; $0.6 * (-100,000) = -60,000$.
 - $\text{EV} = 80,000 + (-60,000) = ₹20,000$.

Interpretation: positive EV (₹20k) — rationally, you should build *if* your probability estimate and payoffs are realistic. But also consider risk tolerance and optionality.

If there's an option to run experiment E (cost = ₹10,000) that will reveal success/failure with some accuracy, compute EV of information and compare $EV(E) - \text{cost}(E)$. If EV of the whole decision after experiment minus cost $>$ EV of acting now, run the experiment.

6) Handling uncertainty & sensitivity analysis

1. **Assign probabilities** to uncertain events — use calibrated ranges (pessimistic/likely/optimistic).
2. **Run sensitivity analysis:** vary key probabilities/payoffs $\pm 30\%$ (or use Monte Carlo for many variables). If ranking flips easily, decision is fragile — pause and run an experiment to reduce uncertainty.
3. **Worst-case / best-case bounds:** calculate min and max outcomes; if min is catastrophic, require guardrails or human-in-loop.
4. **Robustness metrics:** pick option with best *median* and acceptable lower-tail outcome (e.g., 5th percentile).

Quick Monte Carlo outline (conceptual):

- Identify 3 uncertain vars (p_{success} , $\text{revenue_if_success}$, adoption_rate).
 - Sample each variable from a plausible distribution (triangular or beta).
 - Simulate 10k scenarios → derive distribution of outcomes.
 - Use median and lower percentile to decide.
-

7) Decision rules for system design (preserve optionality)

- **Stage-gate:** Do A (discovery experiment) → if metric improves by X% proceed to B (MVP) → if metric improves further proceed to C (scale). Predefine X.
 - **Feature flags:** always deploy behind a flag you can toggle quickly.
 - **Decompose into orthogonal bets:** prefer multiple small bets rather than one big monolithic build.
 - **Rollback plan & budget cap:** set a maximum spend and a rollback criterion before starting.
 - **Human-in-loop thresholds:** require human confirmation if model confidence $< c$ (e.g., 0.75) until model improves.
-

8) Robust decision heuristics (when probabilities are unreliable)

- **Maximin** (choose the option with the best worst-case).
- **Minimax regret** (choose option that minimizes the worst regret across scenarios).

- **Conservative EV:** discount expected payoff by a risk-adjustment factor based on confidence (e.g., multiply EV by 0.7 if low confidence).
- **Portfolio approach:** split budget across several orthogonal small experiments (diversify).

Use these when stakes are high (legal, clinical, reputational) or when models/market are highly unpredictable.

9) Practical templates you can copy

Decision matrix (CSV-style)

```
option,impact(0-10),ease(0-10),risk(0-10),impact_wt,ease_wt,risk_wt,total_score,notes
short_explanations,8,7,3,0.5,0.3,0.2,=0.5*8+0.3*7+0.2*3,"low dev effort"
progressBars,6,9,2,0.5,0.3,0.2,=0.5*6+0.3*9+0.2*2,"very easy"
gamification,9,4,6,0.5,0.3,0.2,=0.5*9+0.3*4+0.2*6,"bigger build"
```

Decision-tree EV sketch (text)

```
DECIDE: Run experiment vs Build
- Experiment (cost 10k) -> 70% yields info that improves p_success estimate and reduces build cost
- Build (cost 100k) -> EV computed as above
Compute EV(experiment) = -10k + [0.7 * EV_with_info + 0.3 * EV_without_info]
Compare to EV(build now). Choose larger.
```

VOI quick step:

1. Compute EV of action now.
2. Compute EV expected after perfect information (EVPI).
3. $EVPI - \text{cost_of_experiment} > 0 \rightarrow \text{run experiment.}$

10) Example applied to your projects

NEETPrepGPT — example quick decision

Problem: add "explanations-short" feature to increase set completion.

Estimate:

- Cost to build = ₹80,000.
- If successful, increases monthly revenue by ₹120,000 (net profit).
- Probability of success $p = 0.35$ (based on current evidence).

Compute EV:

- Net if success = $+120,000 - 80,000 = +40,000$.
- Net if fail = $-80,000$.
- $EV = 0.35 \times 40,000 + 0.65 \times (-80,000)$.
 - $0.35 \times 40,000 = 14,000$.
 - $0.65 \times (-80,000) = -52,000$.
 - $EV = 14,000 - 52,000 = -38,000$ (negative).

Conclusion: don't build now. Run a cheap experiment (wizard-of-oz) costing ₹5,000 to validate uplift. If the experiment improves p to >0.6 , re-evaluate.

Symptom2Specialist — example quick decision

Problem: integrate FHIR now or implement human-in-loop routing first.

Estimate:

- FHIR integration cost = ₹300,000, long partnership time, probability of regulatory hurdles low ($p=0.2$ success to full automation in 3 months).
- Human-in-loop MVP cost = ₹30,000, expected revenue uplift quicker.

Decision: portfolio approach — do human-in-loop first (low cost, fast learning), while starting partnership conversations for FHIR in parallel. Decision matrix will show human-in-loop scores high on ease & time_to_value.

11) Exercises (do these now — timeboxed)

A — 60-minute decision workshop

- Pick 3 candidate features.
- Run a 30-minute decision-matrix with calibrated weights.
- Run a 15-minute sensitivity check on the top option (vary p and payoff $\pm 30\%$).
- Output: chosen option + 48–72 hour experiment to reduce top uncertainty.

B — 2-hour VOI calculation

- For one high-cost decision, estimate probabilities & payoffs; compute EV of acting now vs EV after experiment vs cost of experiment. Decide whether to experiment.

C — 1-day decision-tree build

- Map a choice with two sequential decisions and compute EV at each branch. Use concrete numbers and produce a small T-shaped policy: when to continue, pivot, or stop.

12) Deliverables & mastery criteria

You've mastered Section 6 when you can, in under 48 hours for a non-trivial product decision:

1. Produce a weighted decision matrix with explicit weights and sensitivity column.
2. Compute EV for at least one option with step-by-step math.
3. Run VOI / experiment-cost calculation and recommend either experiment or build.
4. Deliver a stage-gate plan that preserves optionality and includes rollback thresholds.

13) Common failure modes & fixes

- **Overconfident probabilities.** Fix: require evidence & calibration; use historical priors.
- **Ignoring time-to-learn.** Fix: include `time_to_value` as a criterion.
- **Single big bet.** Fix: break into smaller experiments or use portfolio.
- **No explicit rollback.** Fix: require feature flags and budget caps in decision doc.
- **Decisions without owners.** Fix: assign owner, metric, and review date.

Section 7 — Implementation, Scaling, and Monitoring

Short summary: ship small, observe loudly, and design systems so they can be changed cheaply. Implementation is turning decisions into working, testable code. Scaling is making that code work reliably at increasing load. Monitoring is the feedback loop that tells you whether you succeeded and when to pivot.

1) Implementation — plan, owners, and timeboxes

Principles

- Break the chosen solution into **independent, testable increments** that each move the primary metric.
- Ship **observability first** (logs, metrics, traces) for every increment.
- Always deploy behind **feature flags** so you can rollback or limit exposure.
- Default to **human-in-loop** for risky decisions; automate after signal and safety are proven.

Minimal stage-gate plan (for each feature)

1. **Discovery spike (1–3 days)** — prototype “wizard-of-oz” flow, gather signals.
2. **MVP (1–2 weeks)** — minimal automation, high human oversight, basic telemetry.
3. **Validation (2–4 weeks)** — canary/A-B tests, measure metric uplift, collect failure cases.
4. **Harden & scale (2–8 weeks)** — optimize performance, add retries/idempotency, autoscaling.
5. **Automate & retire humans** — only after safety & metrics thresholds met.

Deliverables per increment

- Owner, goal metric, acceptance criteria (numeric), timebox, rollback plan, test plan, telemetry spec.

2) Translating solutions to code — architecture & concrete patterns

High-level architecture patterns

- **Microservice for core logic** (stateless where possible).
- **Data store(s)**: transactional (Postgres-like), cache (Redis), object store (S3).
- **Message bus/queue**: for async work, retries, batching (Kafka/RabbitMQ/Cloud PubSub).
- **Worker pool**: background processing, idempotent workers.
- **API gateway + rate limiting** for external access.
- **Model service**: separate inference service or sidecar with versioning.
- **Feature flags & config service** for runtime control.
- **Observability stack**: metrics (Prometheus), traces (OpenTelemetry), logs (structured JSON to ELK/Graylog/SPLUNK), alerting (Grafana/Alertmanager).

Code organization (one microservice)

```
service/
├─ app/
│   └─ main.py           # FastAPI app + startup hooks
```

```

├── api/
│   ├── v1/
│   │   └── endpoints.py
│   └── core/
│       ├── logic.py
│       ├── jobs/
│       │   └── workers.py
│       ├── db/
│       │   └── models.py
│       ├── schemas/
│       │   └── pydantic_models.py
│       └── metrics.py
├── tests/
├── Dockerfile
├── helm/ k8s/ # deployment manifests or helm chart
└── ci/        # CI workflows

```

Minimal FastAPI example (health + metrics + background task):

```

# app/main.py
from fastapi import FastAPI, BackgroundTasks
from prometheus_client import Counter, generate_latest,
CONTENT_TYPE_LATEST
import time

app = FastAPI()
requests_total = Counter("requests_total", "Total requests")

@app.get("/healthz")
async def health():
    return {"status": "ok", "time": int(time.time())}

@app.get("/metrics")
async def metrics():
    return generate_latest(), 200, {"Content-Type":
CONTENT_TYPE_LATEST}

@app.post("/process")
async def process(payload: dict, background_tasks:
BackgroundTasks):
    requests_total.inc()
    background_tasks.add_task(do_work, payload)
    return {"status": "accepted"}

def do_work(payload):

```

```
# idempotent worker logic, validate input, send to queue or DB  
pass
```

Important implementation details

- **Idempotency:** include message IDs and dedupe logic in workers.
- **Retries:** exponential backoff + max attempts + DLQ.
- **Schema validation:** JSON Schema / Pydantic at the edge.
- **Health & readiness:** `/healthz`, `/readyz` for orchestrator.
- **Config by env:** avoid baked-in secrets; use vault or env-vars.

CI/CD basics

- Run lint → unit tests → integration tests → build image → push → deploy to staging → run smoke tests → promote to canary → monitor → promote to prod.
- Each deployment must be reversible (image tag + feature flag).

Container & deployment snippet (conceptual)

- Dockerfile: small base image, non-root user, healthcheck.
- Kubernetes: Deployment with `readinessProbe`, `livenessProbe`, HPA (cpu/memory), and `PodDisruptionBudget`.

3) Scaling patterns & operational controls

Scaling strategies

- **Horizontal scaling:** add replicas behind load balancer (works for stateless services).
- **Vertical scaling:** increase resources (use sparingly).
- **Caching:** memoize heavy reads (Redis) and cache at CDN edge for static assets.
- **Batching:** accumulate small tasks into bulk jobs to reduce overhead.
- **Sharding / partitioning:** split DB by tenant/key for write scale.
- **Read replicas:** offload analytical reads.
- **Circuit breakers & rate limits:** protect downstream services.

Cost & performance controls

- Set autoscaler targets and max limits.
- Use request quotas and tiered plans to control load from expensive users.
- Monitor cost per request and set alerts on sudden cost spikes.

Data pipelines

- Use checkpoints and watermarking in streaming flows.

- Guarantee at-least-once or exactly-once semantics per use-case; pick dedupe accordingly.

Operational playbooks

- **Canary rollout:** deploy to small % of traffic, watch SLIs, then expand.
- **Blue-green:** for zero-downtime switches when DB migrations are non-breaking.
- **Rollback triggers:** SLI breach for N minutes triggers immediate rollback.

4) Continue vs Pivot — data-driven decision rules

Define before building:

- **Primary metric** (single number) + baseline + target.
- **Decision thresholds** for continue/pivot/kill (e.g., $> +10\%$ uplift $\rightarrow$ continue; $0-10\% \rightarrow$ iterate; $< 0\% \rightarrow$ pivot/kill).
- **Timebox** to collect evidence (e.g., 2–4 weeks or X users).
- **Minimum sample size** and statistical test (A/B test significance) where applicable.

Rules of action

- If metric $>$ target and SLOs hold $\rightarrow$ scale.
- If metric flat but learnings actionable $\rightarrow$ run experiment iteration.
- If metric below kill threshold or safety incident $\rightarrow$ rollback and pause.

Document the **exit criteria** in the stage-gate plan and publish decisions.

5) Monitoring — SLIs, SLOs, alerts, dashboards

Core concepts

- **SLI** (Service Level Indicator): measurable signal (error rate, latency, throughput, data freshness).
- **SLO** (Service Level Objective): target for SLI (e.g., 99.9% success rate).
- **SLA**: contractual agreement (if required).

Minimal monitoring matrix (examples)

- Primary success metric (business): conversion, completion rate, revenue per user.
- SLI-A (availability): HTTP 5xx rate $< 0.1\%$ (SLO 99.9%).
- SLI-B (latency): p95 latency $< 300\text{ms}$ for API (SLO 95%).
- SLI-C (data freshness): max lag < 60 minutes.
- SLI-D (error budget): defined from SLOs and consumed on incidents.

- SLI-E (model health): calibration drift, accuracy by cohort, prediction distribution shifts.

Alerting rules (examples)

- Page if SLI-A breach > 5 minutes.
- Alert if data freshness > threshold or schema validation errors > 0 for 5 min.
- Warn if model confidence distribution shifts 30% from baseline.

Dashboards

- One "launch" dashboard: primary metric + health metrics + model-health panel.
- One infra dashboard: CPU, memory, queue lag, error counts.
- One drift dashboard: input feature distributions, new class frequencies, calibration.

Tracing & logs

- Use distributed tracing (OpenTelemetry) for request flows across services.
- Structured logs with trace_id, request_id, user_id (if permitted), version tags.
- Correlate logs → traces → metrics for rapid RCA.

6) Automated feedback loops (telemetry → action)

Patterns

1. **Data collection:** capture inputs, model outputs, confidences, decisions, ground-truth where available, and user feedback.
2. **Validation & drift detection:** automatic jobs compare current feature distributions to reference and raise alerts (statistical tests, KL divergence).
3. **Retraining triggers:** trigger retrain if performance falls below a threshold or drift exceeds X for Y days.
4. **Human-in-the-loop labeling:** surface low-confidence or disagreement cases to experts for labeling.
5. **Model registry & deployment pipeline:** store model artifact + metadata (version, training data hash, eval metrics). Automate rollout (canary) and rollback.
6. **Post-deploy validation:** monitor shadow predictions vs production decisions and evaluate on labeled samples.

Concrete loop example

- Inference → log (input, prediction, conf, model_version) → if conf < 0.6 send to human review queue → label stored → nightly job re-evaluates model on latest labeled set → if accuracy drop > 3% trigger retrain job (manual approval to deploy).

Governance

- Keep an audit trail (who approved which model version).
 - Keep deletion / data retention policies for privacy compliance.
-

7) Safety, privacy, and compliance (non-negotiables)

Checklist

- Minimize PII collection; store only necessary data.
 - Encrypt data at rest and in transit.
 - Role-based access control; enforce least privilege.
 - Data retention & deletion workflows.
 - Consent capture and user data subject requests process.
 - Logging & auditability for decisions.
 - Bias and fairness checks for models (stratified metrics).
 - Legal review for integrations (FHIR, medical, payment) before live.
-

8) Exercises & sprints (concrete)

Sprint 1 — MVP deploy (2–5 days)

- Implement minimal API + background worker + health endpoints.
- Add Prometheus metrics and `/metrics` endpoint.
- Deploy to staging, run smoke tests.

Sprint 2 — Observability & canary (2–3 days)

- Add distributed tracing and structured logs.
- Implement canary deploy for 5% traffic; monitor SLIs for 24–72 hours.

Sprint 3 — Feedback loop (1–2 weeks)

- Log predictions + confidences; build a small human-review path for low-confidence cases; collect labels.
- Run offline re-eval and document retrain criteria.

Sprint 4 — Scale test & incident playbook (3–5 days)

- Run load test to target expected QPS.
 - Validate autoscaler behavior and queue backpressure.
 - Run simulated incident and follow runbook.
-

9) Deliverables you should produce

1. Implementation checklist & stage-gate doc for the feature.
 2. Repo with code layout, Dockerfile, tests, and CI workflow.
 3. K8s/helm manifests or deployment scripts with health probes and HPA.
 4. Monitoring matrix: SLIs, SLOs, alert rules, dashboard links.
 5. Incident playbook + rollback procedure + on-call contact list.
 6. Feedback-loop spec: what data is collected, where, and retrain triggers.
 7. Model registry entry format and versioning rules.
-

10) Mastery criteria

You've mastered Section 7 when you can, in under 7 days for a new non-trivial feature:

- Deliver an MVP that meets the acceptance criteria and has observability instrumented.
 - Deploy it safely behind a feature flag and run a canary rollout.
 - Collect live telemetry and run at least one retraining candidate pipeline that uses human-labeled data.
 - Run a simulated incident and execute the runbook to mitigate and postmortem.
-

11) Immediate action — do these now (30–120 min)

1. Add `/healthz` and `/metrics` endpoints to your service.
 2. Add request trace IDs to logs and ensure they propagate between services.
 3. Create a one-page monitoring matrix listing primary metric + 3 SLIs + alert thresholds.
 4. Add a feature flag to your next deploy and a rollback plan in your CI workflow.
-

Section 8 — Knowledge Check & Mentorship

Goal: lock the frameworks into reflex by testing, teaching, and running a repeatable mentorship loop so your knowledge compounds when you scale teams and projects.

8.A — Quiz (self-check + group use)

Use this quiz to test retention and provoke applied thinking. Timebox: 45–60 minutes.

Passing: $\geq 80\%$. If below 80%, re-run remediation exercises listed after the answer key.

Format

- 8 short-answer / definition questions (quick recall).

- 4 applied scenario questions (write short plan / run a mini-RCA / design experiment).
- 2 practical tasks to submit (artifact-based).

The Quiz

Short-answer (answer in 1–3 sentences)

1. Define “problem” in the course’s terms (include primary metric).
2. Name the 4 phases of problem solving and a deliverable for each.
3. What is a primary SLI vs an SLO? Give one example for a data pipeline.
4. Explain the 5 Whys in one paragraph and when to stop asking “why”.
5. What does “systems-first thinking” require you to design before code?
6. List three leverage types and one validation step for each.
7. What’s a Wizard-of-Oz test and when should you use it?
8. Give the decision-matrix formula and explain sensitivity.

Applied scenarios (write a short plan — 150–300 words each) 9. You’ve discovered a 40% drop in daily active users after a new release. Outline a 24–72 hour triage + experiment plan (owners, metrics, immediate mitigations). 10. You must choose between building an expensive model or a cheaper human-in-loop flow. Show a decision matrix (3 criteria + weights) and a VOI (value of information) justification for whether to run an experiment first. 11. Design a 7-day Wizard-of-Oz pilot for increasing completion rate of an MCQ set (primary metric, sample size, acceptance range). 12. For an ingestion pipeline showing schema drift, list the fishbone categories and two actionable fixes per category.

Practical submission tasks 13. Upload a filled 5W1H for a live problem (or paste it). 14. Upload (or paste) a one-page system diagram that marks where an AI component would sit, its inputs/outputs, and two monitoring metrics.

Answer key & grading (quick)

- Short answers scored 0–2 each (total 16).
- Applied scenarios scored 0–5 each (total 20).
- Practical tasks scored pass/fail (each 7 points — must meet acceptance criteria).
- Total possible = 50. Passing ≥ 40 .

Remediation: for each missed applied question, do the corresponding sprint (e.g., missed #11 → run the Wizard-of-Oz sprint and report results).

8.B — Mentorship Protocol (how to teach these frameworks so others internalize them)

Purpose: scale competence without diluting quality. Mentorship should create ownership and measurable improvement in mentee capability.

Structure & cadence

- **Onboard (Week 0)** — 1:1, 60–90 min: overview of course, roles, expectations, and one live problem assignment. Deliverables: read the one-page Problem Statement template and fill it within 72 hours.
- **Weekly 1:1s (30–45 min)** — focused on progress, unblockers, and 1 skill exercise (interview, fishbone, mindmap). Mentor gives targeted homework.
- **Biweekly group clinic (60–90 min)** — 3–6 mentees present progress using Six Thinking Hats facilitation; group provides weighted votes and quick experiments.
- **Monthly demo & critique (60–120 min)** — mentee runs a 10–15 min demo of artifact + 30 min critique and acceptance check.
- **Quarterly capstone** — a 2-week mini-project judged against mastery criteria with external reviewer.

Session templates

1:1 (30–45 min)

- 0–5 min: quick metric check (primary metric + delta).
- 5–20 min: problem micro-review (5W1H or experiment outcomes).
- 20–35 min: teach/practice one technique (role-play an interview, run a 5-Why).
- 35–45 min: commitments (deliverable, timebox), risk flag, follow-up date.

Group clinic (90 min)

- 0–10 min: facilitator sets goal & metric.
- 10–30 min: two 10-min presentations (current problem + experiment).
- 30–60 min: Six Hats session (structured ideation).
- 60–80 min: vote and pick experiments.
- 80–90 min: assign owners and publish immediate next steps.

Teaching method: Teach-Do-Review

1. **Teach** — 10–15 min focused explanation with 1 example.
2. **Do** — 20–40 min supervised practice (interview, fishbone, mindmap).
3. **Review** — 10–20 min immediate critique using rubric and re-run parts if needed.

Mentorship feedback rubric (for written artifacts)

- **Clarity (0–4)**: problem & metric clear, numbers present.
- **Evidence (0–4)**: interviews/logs or samples shown; assumptions explicit.
- **Decomposition (0–4)**: sub-problems with acceptance criteria.

- **Experiment design (0–4):** clear owner, metric, sample size, success threshold.
- **Operational thinking (0–4):** monitoring, rollback, privacy considered.
- **Total:** 0–20. Passing for green = ≥ 16 .

Mentor playbook (short)

- Always ask: “How will we know if this worked?” (force a metric).
- Prefer micro-experiments (≤ 2 weeks) over big builds.
- Give feedback that is specific, observable, and prescriptive: “Change X to Y because Z.”
- Track mentee velocity (deliverables per sprint) and learning growth (reduced uncertainty in assumptions).

Microteaching format (for mentees to teach others)

- 10-minute mini-lecture on a single technique (5W1H, 5 Whys, Ishikawa, etc.).
- 20-minute live practice with 2 volunteers.
- 10-minute Q&A and checklist handout. This is how a mentee becomes a mentor.

Scaling mentorship

- Build an **artifact library** (templates, recorded demos, postmortems).
- Use peer review pairs and rotating “lead mentor” role to reduce mentor load.
- Run office-hours (group) instead of endless 1:1s after baseline competency reached.
- Automate intake & tracking with a simple spreadsheet or lightweight LMS (task, owner, due date, score).

Incentives & accountability

- Mentor: recognition, small stipend, or time-credit; expected to invest fixed hours/week.
- Mentee: clear outcomes tied to role or project milestones; public demo required to pass.

8.C — Certification & Mastery (what mentors certify)

To certify someone as “Systems Builder — Level 1” they must **deliver and demonstrate**:

1. One validated Problem Statement with evidence (5W1H + interviews).
2. Pareto table with top 2 causes prioritized and experiments run.
3. One Wizard-of-Oz pilot with data showing either directional uplift or decisive negative result.
4. One system diagram and monitoring matrix.

5. Run a 30–60 minute coached session (teach-do-review) and receive $\geq 16/20$ on rubric for artifacts.

For higher levels (Level 2, 3), require more projects, mentorship hours, and ability to critique and approve others' postmortems and experiments.

Section 9 — Conclusion: Final Word & Next Steps

Short: convert learning into repeated, measurable outputs. Your aim is to turn frameworks into muscle memory and then into organizational capabilities.

9.A — One-page final checklist (do these in order)

1. Pick a live problem (must have measurable primary metric).
 2. Run Section 1–4 sprints: Problem Statement → 4-phase plan → 5W1H → Pareto → Decomposition. (Target: 7–14 days).
 3. Run Section 5 ideation + pick top 2 ideas.
 4. Use Section 6 decision tools to pick experiment(s).
 5. Implement MVP (Section 7) behind flags + instrument.
 6. Run pilot, monitor, and decide continue/pivot.
 7. Produce a postmortem + artifact for Section 8 mentoring.
 8. Repeat, recruit a peer or mentor, and teach the cycle.
-

9.B — 90-day sprint (practical roadmap)

Week 1–2: Problem selection, interviews, 5W1H, Pareto. Week 3–4: Decompose into 6–10 sub-problems, pick top experiment. Week 5–6: Build Wizard-of-Oz and run 7–14 day pilot. Week 7: Analyze results, decide continue/pivot. Week 8–10: If continue, build MVP behind flag, add telemetry. If pivot, run next prioritized experiment. Week 11–12: Harden, write postmortem, mentor one junior on the process.

Measure weekly: primary metric change, number of assumptions validated, artifacts produced.

9.C — Living metrics (what to track continuously)

- **Learning velocity:** completed experiments per month.
- **Assumption reduced:** number of high-uncertainty assumptions tested & resolved.

- **Asset creation:** number of owned leverage assets (datasets, distribution channels).
- **Operational readiness:** % of features with observability + rollback plan.
- **Mentorship growth:** number of mentees certified / mentor hours.

Set targets and review them weekly.

9.D — Final prescriptions (do these now)

- Start with a single real problem and do the **full loop** once (Sections 1→8) in 30 days. That one loop matters more than reading everything.
 - Automate small parts of the loop (templates, dashboards, prompts) so the process scales.
 - Teach one peer within 60 days — teaching forces clarity and reveals gaps.
 - Keep an “Idea Bank” and a “Failed Experiments” log — both are assets.
-